PROJECT 2

DATASET & SCHEMA

BY SARANYA MURALI

TABLE OF CONTENT

TOPICS	PAGE NO
1.INTRODUCTION	3
1.1 WHAT IS DATASET AND TYPES	3
1.2 WHAT IS SCHEMA	4
1.3 COMPONENTS OF SCHEMA	4
1.4 ADVANTAGES OF HAVING SCHEMA	5
1.5 DISADVANTAGES OF HAVING SCHEMA	6
2.DATASET ANALYSIS	6
2.1 TITANIC DATASET	6
2.2 USED CAR SALES DATASET	9
2.3 STUDENT COURSE ENROLMENT DATASET	14
2.4 ADVANTAGES OF NOT HAVING SCHEMA	18
2.5 DISADVANTAGES OF NOT HAVING SCHEMA	18
3.WHAT IF WE DONT FOLLOW PRIMARY KEY RULES	19
4.WHAT IF WE DONT FOLLOW FOREIGNE KEY RULES	20
5.HOW TO DELETE DULICATE DATA	21
6.WHAT IS THE DELETE ORDER WITH PRIMARY AND FOREIGN KEY	22
7.WHAT IS NULL AND NOTNULL CONSTRAIN	23

1.1 What is a DATASET?

A **dataset** is a **collection of related data that is organized in a structured way** so that it can be stored, managed, searched, and analyzed easily.

- Each **row** represents one record.
- Each **column** represents one type of information (called an attribute or field).

Datasets help us to:

- Store information in one place.
- Find information quickly.
- Analyze trends and patterns.
- Generate reports.
- Make business decisions.
- Train Artificial Intelligence (AI) and Machine Learning models.

Types of Datasets

1. Structured Dataset

- Organized into rows and columns.
- Easy to search and analyze.
- Usually stored in databases or Excel files.

2. Semi-Structured Dataset

- Has some organization but not a fixed table format.
- Often stored as JSON or XML files.

Example:

```
{
  "Name": "John",
  "Age": 25,
  "Skills": ["SQL", "Python"]
}
```

3. Unstructured Dataset

- Does not have a predefined structure.
- Includes images, videos, emails, PDFs, audio files, and social media posts.

Examples:

- Photos
- Videos

Common Dataset Formats

- CSV (.csv)
- Excel (.xlsx)
- SQL Database Tables
- JSON (.json)
- XML (.xml)
- Parquet (.parquet)
- Text Files (.txt)

1.2 What is SCHEMA?

A **schema** is the **blueprint or structure of a dataset or database**. It defines **what kind of data can be stored**, **how the data is organized**, and **the rules that the data must follow**.

Simple Example

Imagine creating an Excel sheet for students.
Before entering any student details, you decide:

- Column 1 → Student ID
- Column 2 → Student Name
- Column 3 → Age
- Column 4 → Department
- Column 5 → Marks

This design is called schema.

Schema

Column Name	Data Type	Constraint
StudentID	INT	Primary Key
StudentName	VARCHAR(100)	NOT NULL
Age	INT	Age > 0
Department	VARCHAR(50)	NULL
Marks	DECIMAL(5,2)	NULL

1.3 COMPONENTS OF SCHEMA

- ✓ Table Name
- ✓ Column Name
- ✓ Data Types
- ✓ Primary Key
- ✓ Foreign Keys
- ✓ Constraints
- ✓ Relationships

1.4 Advantages of Having a Schema

Advantage	Explanation
Organized Data	Keeps information in a structured format.
Better Data Quality	Prevents invalid or incorrect data.
Faster Queries	Databases can optimize searches using the schema.
Easy Maintenance	Easier to understand and update.
Security	Access can be controlled at the table or column level.
Consistency	Ensures the same rules apply to all records.
Scalability	Makes it easier to add new features and tables.
Relationships	Connects related tables using keys.

1.5 Disadvantages of Having a Schema

Disadvantage	Explanation
Initial Design Takes Time	The structure must be planned carefully before storing data.
Changes Can Be Difficult	Modifying the schema later may require updates to applications and existing data.
Less Flexible	It can be harder to store unexpected or frequently changing data.

2.DATASET ANALYSIS

Step 1: Understand the Dataset

Step 2: Analyze the Schema

Step 3: Check Data Quality

Step 4: Perform Statistical Analysis

2.1 TITANIC DATASET

Sample

A	B	C	D	E	F	G	H	I	J	K	L
PassengerId	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Survived
1	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25		S	0
2	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85	C	1
3	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925		S	1
4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1	C123	S	1
5	3	Allen, Mr. William Henry	male	35	0	0	373450	8.05		S	0
6	3	Moran, Mr. James	male		0	0	330877	8.4583		Q	0
7	1	McCarthy, Mr. Timothy J	male	54	0	0	17463	51.8625	E46	S	0
8	3	Palsson, Master. Gosta Leonard	male	2	3	1	349909	21.075		S	0
9	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0	2	347742	11.1333		S	1

SQL SCHEMA

```sql
CREATE TABLE TitanicPassengers
(
    PassengerId INT PRIMARY KEY,
    Pclass INT NOT NULL,
    pName VARCHAR(255) NOT NULL,
    Sex VARCHAR(10) NOT NULL,
    Age DECIMAL(5,2) ,
    SibSp INT NOT NULL,
    Parch INT NOT NULL,
    Ticket VARCHAR(50) NOT NULL,
    Fare DECIMAL(10,4) NOT NULL,
    Cabin VARCHAR(20) ,
    Embarked CHAR(1) ,
    Survived BIT NOT NULL
);
```

Column Name	Data Type	Constraint	Description
PassengerId	INT	PRIMARY KEY	Unique passenger identifier
Pclass	INT	NOT NULL	Passenger class (1, 2, or 3)
Name	VARCHAR(255)	NOT NULL	Passenger's full name
Sex	VARCHAR(10)	NOT NULL	Gender (Male/Female)
Age	DECIMAL(5,2)	NULL	Passenger age (can be missing)
SibSp	INT	NOT NULL	Number of siblings/spouses aboard
Parch	INT	NOT NULL	Number of parents/children aboard
Ticket	VARCHAR(50)	NOT NULL	Ticket number
Fare	DECIMAL(10,4)	NOT NULL	Ticket fare
Cabin	VARCHAR(20)	NULL	Cabin number (may be missing)
Embarked	CHAR(1)	NULL	Port of embarkation (C, Q, S)
Survived	BIT	NOT NULL	Survival status (0 = No, 1 = Yes)

Relationships Between Entities

The Titanic dataset contains only one entity, which is Passenger. Therefore, there are no foreign key relationships because all passenger information is stored on a single table.

Related Attributes	Relationship
PassengerId → Passenger	Uniquely identify each passenger.
Pclass → Fare	First-class passengers generally paid higher fares than second- or third-class passengers.

Primary Key: `PassengerId`

Foreign Keys: None

Reason:

- Each passenger has a unique `PassengerId`.
- The dataset consists of a single table, so no foreign keys are needed.

2.1.1Advantages of Having a Schema

Advantage	Explanation	Titanic Dataset Example
Data Consistency	Ensures every record follows the same structure.	Every passenger has columns like `PassengerId`, `Name`, `Age`, and `Fare`.
Improved Data Quality	Prevents incorrect or invalid data from being entered.	`Age` is stored as a numeric value instead of text such as **"Twenty".**
Faster Data Processing	Applications know the structure beforehand, making processing more efficient.	SQL can quickly calculate the average fare or passenger count.
Efficient Data Retrieval	Queries can locate records quickly using keys and indexed columns.	Find a passenger using `PassengerId`.
Easy Data Analysis	Data visualization and reporting tools can understand the dataset easily.	Create charts showing survival rates by gender or passenger class.
Reduced Data Redundancy	Standardized columns reduce duplicate or inconsistent information.	All passenger names are stored in a single `Name` column.
Better Maintenance	Developers understand the dataset structure more easily.	New analysts can immediately understand the meaning of each column.

2.1.2 Disadvantages of Having a Schema

Disadvantage	Explanation	Example
Less Flexible	Adding new columns or changing existing ones requires modifying the schema.	Adding a new `Nationality` column requires updating the table structure.

Initial Design Effort	Designing proper schema takes time and planning.	Deciding the correct data type for each column before storing data.
Schema Changes Can Be Costly	Existing applications and reports may need updates when the schema changes.	Renaming `Fare` to `TicketFare` could affect SQL queries and dashboards.
Complexity for Large Systems	Large databases may have many related tables and relationships.	Airlines or banking systems may contain hundreds of interconnected tables.

2.1.3 PRACTICAL EXAMPLE

Scenario 1 – Calculating Average Age

Without Schema

Name	Age
John	Twenty
David	25
Sara	Thirty

Average age cannot be calculated correctly.

With Schema

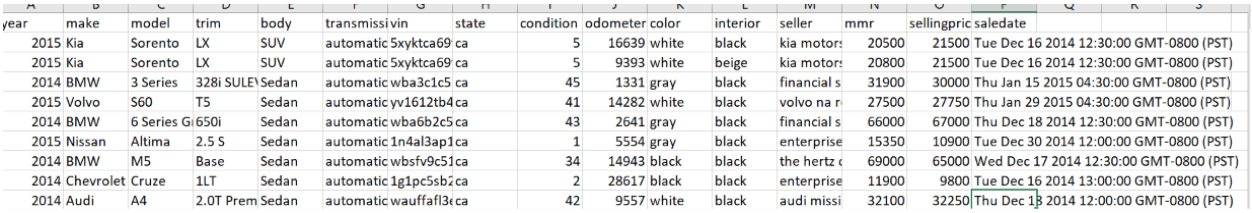
Name	Age
John	20
David	25
Sara	30

Average Age

(20 + 25 + 30) / 3 = 25

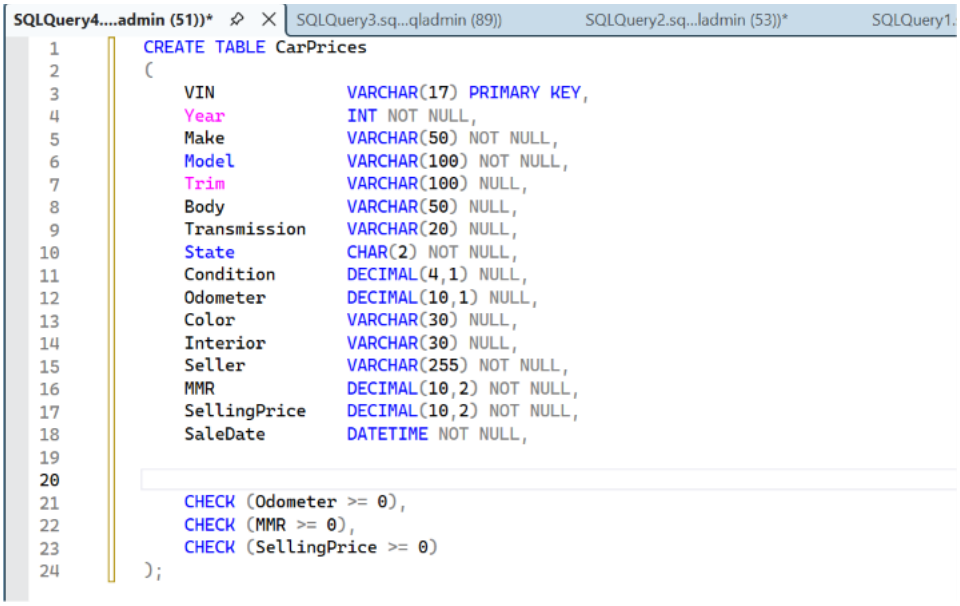
2.2 USED CAR SALES DATASET

Sample



year	make	model	trim	body	transmissi	vin	state	condition	odometer	color	interior	seller	mmr	sellingpric	saledate
2015	Kia	Sorento	LX	SUV	automatic	5xyktca69	ca	5	16639	white	black	kia motors	20500	21500	Tue Dec 16 2014 12:30:00 GMT-0800 (PST)
2015	Kia	Sorento	LX	SUV	automatic	5xyktca69	ca	5	9393	white	beige	kia motors	20800	21500	Tue Dec 16 2014 12:30:00 GMT-0800 (PST)
2014	BMW	3 Series	328i SULE	Sedan	automatic	wba3c1c5	ca	45	1331	gray	black	financial s	31900	30000	Thu Jan 15 2015 04:30:00 GMT-0800 (PST)
2015	Volvo	S60	T5	Sedan	automatic	yv1612tb4	ca	41	14282	white	black	volvo na r	27500	27750	Thu Jan 29 2015 04:30:00 GMT-0800 (PST)
2014	BMW	6 Series Gi	650i	Sedan	automatic	wba6b2c5	ca	43	2641	gray	black	financial s	66000	67000	Thu Dec 18 2014 12:30:00 GMT-0800 (PST)
2015	Nissan	Altima	2.5 S	Sedan	automatic	1n4al3ap1	ca	1	5554	gray	black	enterprise	15350	10900	Tue Dec 30 2014 12:00:00 GMT-0800 (PST)
2014	BMW	M5	Base	Sedan	automatic	wbsfv9c51	ca	34	14943	black	black	the hertz c	69000	65000	Wed Dec 17 2014 12:30:00 GMT-0800 (PST)
2014	Chevrolet	Cruze	1LT	Sedan	automatic	1g1pc5sb2	ca	2	28617	black	black	enterprise	11900	9800	Tue Dec 16 2014 13:00:00 GMT-0800 (PST)
2014	Audi	A4	2.0T Prem	Sedan	automatic	wauffafl3	ca	42	9557	white	black	audi missi	32100	32250	Thu Dec 18 2014 12:00:00 GMT-0800 (PST)

Schema



SQLQuery4....admin (51))* | SQLQuery3.sq...qladmin (89)) | SQLQuery2.sq...ladmin (53))* | SQLQuery1.

```
CREATE TABLE CarPrices
(
    VIN             VARCHAR(17) PRIMARY KEY,
    Year            INT NOT NULL,
    Make            VARCHAR(50) NOT NULL,
    Model           VARCHAR(100) NOT NULL,
    Trim            VARCHAR(100) NULL,
    Body            VARCHAR(50) NULL,
    Transmission    VARCHAR(20) NULL,
    State           CHAR(2) NOT NULL,
    Condition       DECIMAL(4,1) NULL,
    Odometer        DECIMAL(10,1) NULL,
    Color           VARCHAR(30) NULL,
    Interior        VARCHAR(30) NULL,
    Seller          VARCHAR(255) NOT NULL,
    MMR             DECIMAL(10,2) NOT NULL,
    SellingPrice    DECIMAL(10,2) NOT NULL,
    SaleDate        DATETIME NOT NULL,

    CHECK (Odometer >= 0),
    CHECK (MMR >= 0),
    CHECK (SellingPrice >= 0)
);
```

Schema Details

Column Name	Data Type	Constraint	Description
VIN	VARCHAR(17)	PRIMARY KEY, NOT NULL	Unique Vehicle Identification Number

Year	INT	NOT NULL, CHECK	Manufacturing year of the vehicle
Make	VARCHAR(50)	NOT NULL	Vehicle manufacturer
Model	VARCHAR(100)	NOT NULL	Vehicle model
Trim	VARCHAR(100)	NULL	Vehicle trim or variant
Body	VARCHAR(50)	NULL	Body style (SUV, Sedan, Coupe, etc.)
Transmission	VARCHAR(20)	NULL	Automatic or Manual transmission
State	CHAR(2)	NOT NULL	State where the vehicle was sold
Condition	DECIMAL(4,1)	CHECK (0–5)	Vehicle condition rating
Odometer	DECIMAL(10,1)	CHECK (>=0)	Distance traveled
Color	VARCHAR(30)	NULL	Exterior color
Interior	VARCHAR(30)	NULL	Interior color
Seller	VARCHAR(255)	NOT NULL	Dealer or seller name
MMR	DECIMAL(10,2)	NOT NULL, CHECK (>=0)	Estimated market value
SellingPrice	DECIMAL(10,2)	NOT NULL, CHECK (>=0)	Actual selling price
SaleDate	DATETIME	NOT NULL	Date and time of sale

Relationships Between Attributes

Since this dataset has only one table, there are **no foreign key relationships**. However, several attributes are logically related.

Related Attributes	Relationship
VIN → Vehicle	VIN uniquely identifies each vehicle.
Year → Selling Price	Newer vehicles generally sell for higher prices.

Current Dataset

Table: CarPrices
Primary Key: VIN
Foreign Keys: None

Possible Foreign Keys After Normalization

Instead of storing everything in one table, you can create multiple tables.

VIN can be considered as a foreign key in different tables.

Sellers Table

SellerID (PK)	SellerName	VIN (FK)
101	ABC Motors	ABC123
102	XYZ Auto Sales	XYZ456

Foreign Key

2.2.1. Advantages of Having a Schema

Advantage	Explanation	Example
Data Consistency	Every vehicle follows the same structure.	Every record contains `Year`, `Make`, `Model`, and `SellingPrice`.
Better Data Quality	Prevents incorrect values.	`SellingPrice` is always numeric.
Faster Processing	Systems know each column's data type.	Average selling price is calculated quickly.
Efficient Storage	Numeric columns require less storage than text.	Odometer stored as numbers instead of strings.
Easy Analysis	BI tools understand the dataset automatically.	Create reports by make, model, or state.
Improved Retrieval	Searching and filtering become faster.	Find a car using its VIN.

2.2.2 Disadvantages of Having a Schema

Disadvantage	Explanation
Less Flexible	Adding new columns requires changing the schema.
Initial Design Time	The schema must be designed before loading data.
Maintenance	Applications may need updates when the schema changes.

Large Dataset Complexity	Large datasets require careful schema management.

Practical Scenarios

2.2.3 Scenario 1 – Vehicle Search

Without Schema

Vehicle_ID
VIN_Number
Car_ID

Searching becomes inconsistent.

With Schema

`VIN`

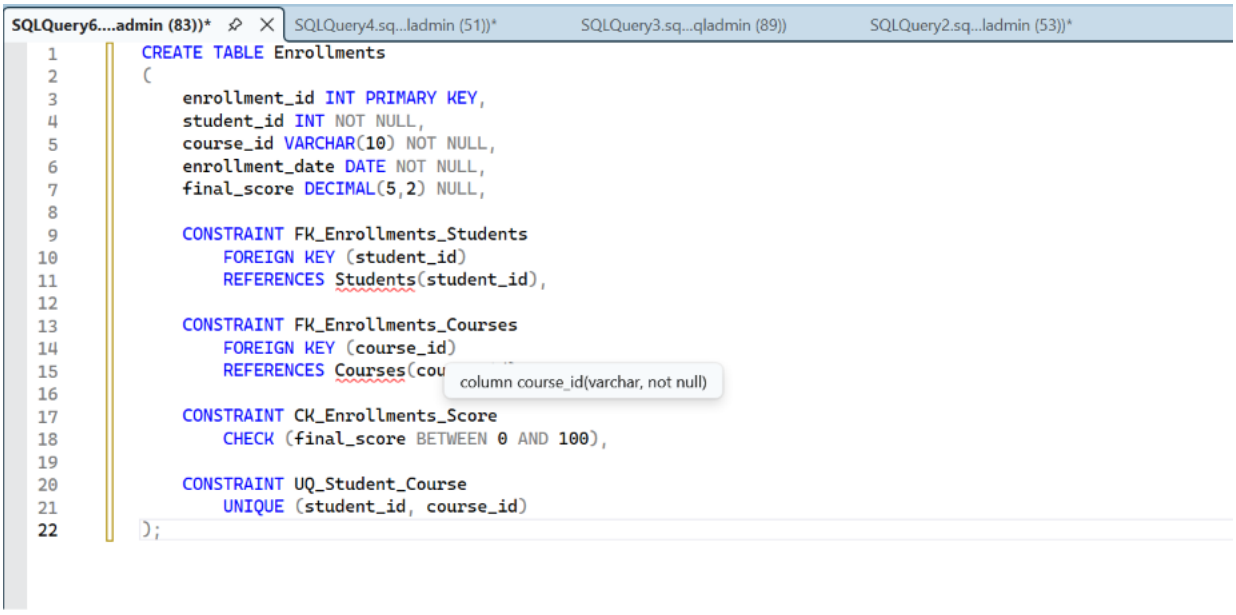
Every system uses the same identifier.

2.3 STUDENT COURSE ENROLMENT DATASET

Sample

enrollment_id	student_id	student_name	course_id	enrollment_date	final_score
1	101	John Smith	C101	2026-01-10	85
2	101	John Smith	C103	2026-02-01	90
3	102	Priya Kumar	C102	2026-01-15	95
4	103	David Lee	C101	2026-01-20	78
5	103	David Lee	C104	2026-03-01	82
6	104	Sara Ahmed	C103	2026-02-10	88
7	105	Michael Brown	C102	2026-02-15	75

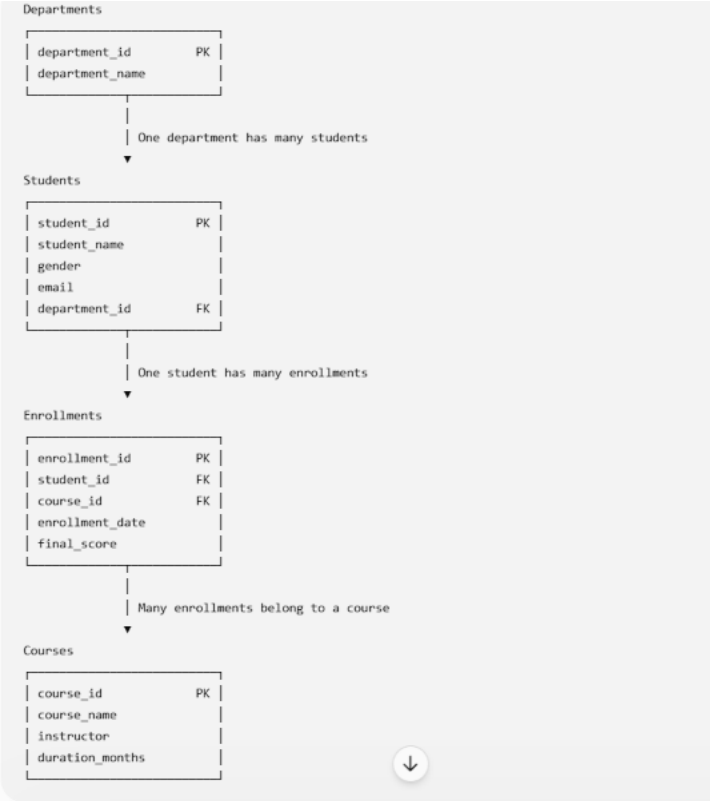
Schema



Schema

Column Name	Data Type	Constraint	Description
enrollment_id	INT	**PRIMARY KEY, NOT NULL**	Unique enrollment record ID
student_id	INT	**NOT NULL, FOREIGN KEY**	References `Students(student_id)`
course_id	VARCHAR(10)	**NOT NULL, FOREIGN KEY**	References `Courses(course_id)`
enrollment_date	DATE	**NOT NULL**	Date when the student enrolled
final_score	DECIMAL(5,2)	**CHECK (0–100)**	Student's final score
grade	CHAR(1)	**CHECK ('A','B','C','D','F')**	Letter grade

RELATIONSHIP DIAGRAM



Relationships

Parent Table	Primary Key	Child Table	Foreign Key	Relationship
Students	`student_id`	Enrollments	`student_id`	One-to-many
Courses	`course_id`	Enrollments	`course_id`	One-to-many

2.3.1 Advantages of the Schema

Advantage	Explanation	Example
1. Eliminates Data Redundancy	Student and course information are stored only once, reducing duplicate data.	"John Smith" is stored once in the `Students` table instead of being repeated in every enrollment record.

2. Maintains Data Integrity	Foreign keys ensure that only valid students and courses can be added to the `Enrollments` table.	A student cannot enroll in a course unless both the student and the course exist.
3. Prevents Duplicate Enrollments	The `UNIQUE(student_id, course_id)` constraint prevents a student from enrolling in the same course more than once.	Student 101 cannot enroll in SQL Fundamentals twice.
4. Improves Data Accuracy	`CHECK` constraints ensure valid values are stored.	`final_score` must be between 0 and 100, and `grade` must be A, B, C, D, or F.
5. Efficient Data Retrieval	Primary and foreign keys enable efficient joins between related tables.	Retrieve a student's name, department, and enrolled courses using SQL JOINs.
6. Easier Maintenance	Updating information in one place automatically reflects across related records.	If a student's email changes, it is updated only in the `Students` table.
7. Better Security	Sensitive information can be stored separately and access can be restricted by table.	Student personal details can be protected while allowing instructors to view only enrollments.

2.3.2 Disadvantages of the Schema

Disadvantage	Explanation	Example
1. More Complex Design	Multiple related tables make the database structure more complex than a single table.	Understanding relationships between Students, Courses, and Enrollments requires knowledge of joins.
2. Requires SQL JOINs	Retrieving complete information requires joining multiple tables.	To display a student's department and course, several tables must be joined.
3. Slightly Slower Queries	Complex joins may reduce performance with very large datasets.	Reports involving thousands of students and courses may take longer than querying a single table.

4. More Tables to Maintain	Database administrators need to manage several tables and relationships.	Adding new features may require changes to multiple tables.
5. Data Entry Order Matters	Parent records must exist before child records can be inserted.	A student must be created in the `Students` table before adding an enrollment record.
6. Constraint Violations	Invalid inserts or updates are rejected by the database.	Attempting to enroll `student_id = 999` fails because that student does not exist.

2.4 ADVANTAGES OF NOT HAVING SCHEMA

Although having a schema is generally recommended, there are some situations where not having a predefined schema can be beneficial.

Advantage	Explanation	Example
Greater Flexibility	New fields can be added without changing the database structure.	A student dataset can easily add a new column such as `ScholarshipStatus` without redesigning the schema.
Faster Initial Development	Developers can start storing data without spending time designing a schema.	During a prototype, student records can be stored

		immediately without creating tables first.
Suitable for Unstructured Data	Ideal for storing documents, images, logs, JSON, or social media data where every record may have different fields.	A student feedback system where every feedback form contains different questions.
Easier to Handle Changing Requirements	If business requirements change frequently, the database can adapt without schema modifications.	A university frequently adds new student attributes each semester.
Supports Rapid Data Collection	Data from different sources can be stored without first converting it into a fixed format.	Importing survey responses with varying numbers of questions.

2.5 DISADVANTAGES OF NOT HAVING SCHEMA

Disadvantage	Explanation	Example
Poor Data Quality	Invalid or inconsistent data can be stored because there are no rules.	Age may be entered as `"Twenty"` instead of `20`.
Data Inconsistency	Different records may use different column names for the same information.	One record uses `StudentName`, another uses `Name`, and another uses `FullName`.
Duplicate Data	The same information may be stored repeatedly.	A student's department is repeated in every course enrollment record.
4. Difficult Data Analysis	Inconsistent data makes reporting and analysis more difficult.	Calculating the average age is impossible if ages are stored as both numbers and text.

3.WHAT IF WE DONT FOLLOW PRIMARY KEY RULES

A **Primary Key** is used to uniquely identify each record in a table. If primary key rules are not followed, the database can become inconsistent, unreliable, and difficult to manage.

Consequence of not having primary key

- ✓ Duplicate Records
- ✓ Data Inconsistency

- ✓ Difficulty Retrieving Data
- ✓ Referential Integrity Issues
- ✓ Update Anomalies
- ✓ Delete Anomalies
- ✓ Insertion Errors
- ✓ Poor Data Quality

Example

Duplicate Primary Key

Students Table

student_id	student_name	Department
101	John Smith	Computer Science
101	David Lee	Information Technology

Problems

- Which student is 101?
- Queries return multiple results.
- Updates and deletes become unreliable.

A **Primary Key** is one of the most important constraints in a relational database. If primary key rules are not followed, duplicate records, NULL values, incorrect relationships, update anomalies, and data inconsistency can occur. This affects data quality, retrieval, reporting, and overall database reliability. Therefore, every table should have a properly defined primary key to ensure that each record is uniquely and accurately identified.

4.WHAT IF WE DONT FOLLOW FOREIGNE KEY RULES

A Foreign Key (FK) is a column in one table that references the Primary Key (PK) of another table. It ensures that relationships between tables remain valid and consistent.

CONSEQUENCES

A Foreign Key ensures that relationships between tables remain valid by requiring every child to record an existing parent record. If foreign key rules are not followed, the database may contain orphan records, broken relationships, inaccurate reports, update

anomalies, and poor data integrity. Enforcing foreign key constraints helps maintain consistency, reliability, and accuracy in relational databases.

Real-World Scenario

Imagine a **university system**.

Students

Student ID	Name
101	John
102	Priya

Enrollments

Student ID	Course
999	SQL

The university report shows: "Student 999 is enrolled in SQL."
But Student 999 doesn't exist.
This creates inaccurate reports and confusion.

5.HOW TO DELETE DULICATE DATA

Method 1: Using CTE and ROW_NUMBER()

```
WITH CTE_DuplicateRecords AS
(
   SELECT *,
        ROW_NUMBER() OVER
        (
           PARTITION BY StudentID
           ORDER BY StudentID
        ) AS RowNum
```

```
    FROM Students
)
DELETE FROM CTE_DuplicateRecords
WHERE RowNum > 1;
```

Method 2: Using GROUP BY

```
SELECT StudentID,
        COUNT(*) AS DuplicateCount
FROM Students
GROUP BY StudentID
HAVING COUNT(*) > 1;
```

This query finds duplicates but does not delete them.

6.WHAT IS THE DELETE ORDER WITH PRIMARY AND FOREIGN KEY

When tables are related using primary and foreign keys, records should always be deleted from the child table first, followed by the parent table. This prevents foreign key constraint violations and maintains referential integrity. If `ON DELETE CASCADE` is configured, deleting the parent record automatically deletes the related child records.

Example

Parent Table: Students

student_id (PK)	student_name
101	John
102	Priya

Child Table: Enrollments

enrollment_id	student_id (FK)	course
1	101	SQL
2	101	Python
3	102	Power BI

Here:

- Students.student_id is the **Primary Key**.
- Enrollments.student_id is the **Foreign Key**.

Correct Delete Order

Step 1: Delete Child Records

```
DELETE FROM Enrollments
WHERE student_id = 101;
```

Step 2: Delete Parent Record

```
DELETE FROM Students
WHERE student_id = 101;
```

Now the delete succeeds because there are no child records referencing Student 101.

7.WHAT IS NULL AND NOTNULL CONSTRAIN

NULL and NOT NULL are database constraints that define whether a column is allowed to store an empty value.

What is a NULL Constraint?

A NULL constraint means the column is allowed to have no value.

It indicates that the information is unknown, unavailable, or not applicable.

Simple Definition

NULL means a column can store an empty or missing value.

Example

Student Table

StudentID	StudentName	PhoneNumber
101	John	9876543210
102	Priya	NULL
103	David	9123456789

HERE THERE IS NO PHONE NUMBER FOR STUDENT PRIYA SO ITS ENTERED AS **NULL.**

Advantages of NULL

- Optional information can be left empty.
- Useful when data is not available.
- Makes the database flexible.

Disadvantages of NULL

- Queries become more complex.
- Aggregate functions may ignore NULL values.
- Can make data analysis more difficult.

What is a NOT NULL Constraint?

A NOT NULL constraint means every record must contain a value.

The database will not allow empty values in that column.

Example

Student Table

StudentID	StudentName	Email
101	John	john@email.com
102	Priya	priya@email.com

Every student must have an email.

NULL vs NOT NULL

NULL	NOT NULL
Value can be empty	Value is mandatory
Optional information	Required information
Allows missing data	Does not allow missing data
More flexible	Better data quality
Example: Cabin Number	Example: Student ID

Advantages of NOT NULL

- Improves data quality.
- Prevents missing important information.
- Ensures mandatory fields are always completed.
- Makes reporting more reliable.

- Simplifies application logic because required values are guaranteed.

Disadvantages of NOT NULL

- Less flexible.
- Users must provide a value before inserting a record.
- Temporary or incomplete records cannot be stored without placeholder values.